

Short Order Arena - CPU Scheduling Project

Quick Summary for AI Assistants

I'm working on a university CPU scheduling assignment called "Short Order Arena." I need to write a Python scheduler that scores above 71.0 (the top reference scheduler called "gordon_ramsay"). My current best score is **68.7 +/- 0.9**. I need help improving the scheduler to beat 71.0.

1. Project Overview

Course: Operating Systems (University of the Witwatersrand) **Assignment:** Write a CPU scheduler disguised as a restaurant kitchen simulation **File:** One Python file: `my-scheduler/scheduler.py` (max 256 KB) **Deadline:** Submit to Moodle

The Metaphor

Kitchen Concept	CPU Concept
Cooks	CPU cores
Orders	Processes
Recipe work steps (chop, sear)	CPU bursts
Recipe wait steps (oven, simmer)	I/O bursts
Context switch cost	<code>switch_cost</code> ticks
Customer patience	Deadline
Stations (bench, pass, bar)	Semaphores with limited permits
Ticket rail	Ready queue

Constraints

- `schedule()` function has **50ms per call** (timeout = discarded)
 - 20 consecutive failures = **forfeit** (score 0)
 - Single file only, no extra imports (subprocess/socket/pickle banned)
-

2. The Interface

```
from kitchen import Decision, Scheduler, fill_idle

class MyScheduler(Scheduler):
    name = "my_scheduler"
    version = "1"

    def reset(self, seed):
        """Called once before each run."""
        pass

    def schedule(self, obs):
        """Called at every scheduling point. Return a Decision."""
        return Decision()
```

Key Observation API (obs)

obs.time	Current tick
obs.kitchen.cores	Number of cooks
obs.kitchen.switch_cost	Ticks lost per context switch
obs.kitchen.known_durations	Whether step durations are visible
obs.ready	Orders waiting for a cook (arrival order)
obs.runnable	Ready orders whose station has a free place
obs.idle_cores	Cooks with nothing to do
obs.working_cores	Cooks currently working
obs.stations	Each station: name, capacity, busy, free
obs.free_stations()	Dict of {name: places_free}
obs.order(id)	Look up an order by id
obs.estimate_remaining(o)	Estimated work ticks remaining

Key Order Properties

order.id	Unique identifier
order.recipe	Recipe name (espresso, burger, etc.)
order.priority	1=regular, 2=hurried, 3=VIP
order.arrival	Tick when order arrived
order.deadline	Tick when customer walks out
order.time_left	deadline - now
order.work_remaining	Ticks of work left (None if hidden)
order.work_until_wait	Ticks of work before next oven step
order.station	Station needed for current step
order.waited	Ticks spent on the rail
order.steps	List of Step objects (label, kind, station, duration)

Decision API

```
d = Decision()
d.assign(core, order)    # Put order on cook (preempts current)
d.idle(core)             # Take cook off order
d.wake_in(ticks)         # Timer interrupt - call me again in N ticks
d.annotate(text, queue)  # Debug visualization (ignored by engine)
```

Helper Function

```
fill_idle(decision, obs, queue) # Assign idle cores to orders in queue order
                                # Returns remaining unassigned orders
                                # Handles station capacity automatically
```

3. Scoring System (6 Components, Weighted to 100)

Component	Weight	Formula	What It Measures
Completion	30	served / total_orders	How many customers served
Response	12.5	$1 / (1 + \text{mean_response} / 25)$	Time to first cook

Component	Weight	Formula	What It Measures
Turnaround	12.5	$1 / (1 + \text{mean}(\text{left} - \text{arrived}) / 60)$	Total time in system
Slowdown	15	$1 / \text{mean}(\text{turnaround} / \max(\text{span}, 10))$	Relative delay
Switching	15	$\text{necessary_switches} / \text{all_switches}$	Minimize preemptions
Fairness	15	Jain's index over slowdowns	Even distribution

$$\text{Score} = 100 \times \text{sum}(\text{component} \times \text{weight}) / \text{sum}(\text{weights})$$

Critical Scoring Insights

- **Completion is king** (30% weight) - abandoned orders hurt twice (lost customer + wasted cook time)
- **Slowdown** = turnaround / max(order's total span, 10). Orders needing <10 ticks count as 10
- **Fairness** uses Jain's index: 1.0 = perfect equality, lower = more variance
- **Switching**: "necessary" = first time working on an order. "unnecessary" = resumption after preemption
- A customer who walks out STILL counts for response, turnaround, slowdown, fairness

4. Reference Schedulers (Built Into Engine)

Name	Algorithm	Score	Served
gordon_ramsay	Unknown (best reference)	71.0	95.2%
julia_child	SJF (Shortest Job First)	68.4	94.4%
sous_chef	SRTF (Shortest Remaining Time)	64.5	93.3%
portion_control	Stride (proportional share)	64.2	81.3%
trainee	EDF (Earliest Deadline First)	58.1	89.5%
maitre_d	Random	58.0	70.8%
smoke_break	MLFQ (Multi-Level Feedback)	57.5	76.4%
middle_management	FIFO	57.3	71.6%
stickler	Priority	55.3	79.6%
headless_chicken	Round Robin (8-tick slices)	51.9	63.5%
idiot_sandwich	Longest Job First (starter)	49.2	49.2%
dishwasher	Never cooks	0.0	0.0%

gordon_ramsay's Metrics (The Target)

Score: 71.0 +/- 0.9
 Completion: 95.2%
 Response: 35.3 ticks (mean)
 Turnaround: 65.2 ticks (mean)
 Slowdown: 1.79 (bounded)
 Switches: 390.7 total, 96% necessary (15.9 preemptions/run)
 Fairness: 0.558 (Jain's index)

5. Marking Profiles (9 Total)

Your final mark = mean score across ALL profiles.

Profile	Description	Key Challenge
default	“Lunch service” - standard	General scheduling
rush	Heavy load, more customers	Prioritization under pressure
bake-off	I/O-bound (lots of oven time)	I/O overlap
banquet-night	Long + short jobs mixed	Convoy effect
blind	Durations hidden	Estimation quality
one-cook	Single core	Classic SJF/EDF
brigade	4 cooks	Load balancing
service-line	4 cooks, 1 pass station	Bottleneck management
function	All orders arrive at once	Timer-based scheduling (wake_in)

6. My Current Best Scheduler (Score: 68.7)

```
from kitchen import Decision, Scheduler, fill_idle

class SmartScheduler(Scheduler):
    name = "smart_scheduler"
    version = "final"

    def reset(self, seed):
        self.profile_type = None

    def schedule(self, obs):
        decision = Decision()
        est = obs.estimate_remaining
        switch_cost = obs.kitchen.switch_cost

        # Detect profile on first call
        if self.profile_type is None:
            name = obs.kitchen.name.lower()
            if "function" in name:
                self.profile_type = "function"
            elif "service" in name and "line" in name:
                self.profile_type = "service_line"
            elif "blind" in name:
                self.profile_type = "blind"
            elif "bake" in name:
                self.profile_type = "bake_off"
            elif "brigade" in name:
                self.profile_type = "brigade"
            elif "banquet" in name:
                self.profile_type = "banquet"
            elif "one" in name:
                self.profile_type = "one_cook"
            elif "rush" in name:
                self.profile_type = "rush"
            else:
```

```

        self.profile_type = "default"

# Deadline feasibility filter
feasible = []
for order in obs.ready:
    remaining = est(order)
    if order.time_left >= remaining + switch_cost:
        feasible.append(order)

# Scoring function
def sort_key(order):
    remaining = est(order)
    waited = order.waited
    urgency = remaining / max(1, order.time_left)
    score = remaining + urgency * 20

    if self.profile_type == "bake_off":
        wuw = order.work_until_wait
        if wuw is not None and wuw < remaining and wuw > 0:
            score -= (remaining - wuw) * 0.3

    if order.priority >= 3:
        score -= 5
    elif order.priority >= 2:
        score -= 2

# Adaptive aging
    if self.profile_type == "banquet":
        score -= waited * 0.04
    elif self.profile_type in ("blind", "rush"):
        score -= waited * 0.03
    else:
        score -= waited * 0.02

    return (score, order.arrival)

queue = sorted(feasible, key=sort_key)
rest = fill_idle(decision, obs, queue)

# Timer for function profile
if self.profile_type == "function" and obs.working_cores:
    decision.wake_in(8)

# Preemption
if self.profile_type == "one_cook":
    preempt_thresh = 9999
elif self.profile_type == "brigade":
    preempt_thresh = 35
else:
    preempt_thresh = 25

if preempt_thresh < 9999:
    free = obs.free_stations()

```

```

for oid in decision.assignments.values():
    if oid is None: continue
    o = obs.order(oid)
    if o and o.station is not None:
        free[o.station] = free.get(o.station, 0) - 1

for core in obs.working_cores:
    if not rest: break
    current = obs.order_on(core)
    if current is None: continue
    best_idx = None
    for i, oid in enumerate(rest):
        c = obs.order(oid)
        if c and (c.station is None or free.get(c.station, 0) > 0):
            best_idx = i
            break
    if best_idx is None: continue
    candidate = obs.order(rest[best_idx])
    if candidate is None: continue
    if est(current) - est(candidate) > preempt_thresh:
        decision.assign(core, rest.pop(best_idx))
        if candidate.station is not None:
            free[candidate.station] = free.get(candidate.station, 0) - 1

return decision.annotate(
    text=f"{self.profile_type}: {len(queue)}/{len(obs.ready)}",
    queue=[o.id for o in queue],
)

```

My Scheduler's Metrics

Score: 68.7 +/- 0.9
 Completion: 94.9%
 Response: 31.8 ticks (BETTER than gordon_ramsay!)
 Turnaround: 63.6 ticks (BETTER than gordon_ramsay!)
 Slowdown: 1.92 (bounded)
 Switches: 401.1 total, 95% necessary (22.4 preemptions/run)
 Fairness: 0.435 (WORSE - this is the main gap)

7. Gap Analysis: Where I'm Losing to gordon_ramsay

Metric	Mine	gordon_ramsay	Gap	Impact
Completion	94.9%	95.2%	-0.3%	~0.1 points lost
Response	31.8	35.3	+3.5 better	I'm WINNING here
Turnaround	63.6	65.2	+1.6 better	I'm WINNING here
Slowdown	1.92	1.79	+0.13 worse	~1.0 point lost
Necessary%	95%	96%	-1%	~0.2 points lost
Fairness	0.435	0.558	-0.123	~1.8 points lost

Total gap: ~2.3 points (68.7 vs 71.0)

The Fairness Problem

Fairness (Jain's index) is the SINGLE BIGGEST gap. My scheduler starves long jobs too much because SJF naturally prioritizes short jobs. To improve fairness, I need to serve long-waiting orders sooner, but that hurts response time and slowdown.

This is the core tradeoff I need help solving.

8. What I've Already Tried (30+ Configurations)

Approach	Score	Notes
Pure SJF, no preemption	67.3	Good baseline
SJF + preemption (threshold 15)	67.5	Slightly better
SJF + preemption (threshold 25)	67.9	Best preemption
SJF + aging (0.02 per tick)	67.5	Light aging helps slightly
SJF + aging (0.05 per tick)	66.5	Too much aging hurts
SJF + aging (0.1 per tick)	65.0	Much worse
Urgency (w=10) + aging (0.02)	68.2	Good
Urgency (w=15)	68.4	Better
Urgency (w=20) + aging (0.02)	68.7	BEST
Urgency (w=30)	68.5	Diminishing returns
HRRN (Highest Response Ratio Next)	57.2	Terrible - starves short jobs
Stride scheduling	68.5	Similar to urgency
Two-level priority	62-65	Hurts completion
Deadline-first sorting	68.3	Good response, bad completion
Aggressive aging (0.05-0.1)	65-67	Better fairness but worse overall
I/O overlap bonus	65.5	Hurt on default profile
sqrt(waited) aging	67.7	Not enough improvement
waited^1.5 aging	59.6	Way too aggressive

Key Findings

1. **Pure SJF with light aging is near-optimal for greedy approaches**
 2. **Urgency weighting (remaining/time_left) significantly helps response/slowdown**
 3. **Aggressive aging improves fairness but hurts overall score**
 4. **Preemption helps when threshold is 20-30 (not too aggressive)**
 5. **The fundamental SJF-vs-fairness tradeoff seems to cap greedy approaches at ~69**
-

9. Ideas to Explore (Need Help With These)

A. Multi-Level Feedback Queue (MLFQ)

- Multiple priority queues with different quantum lengths
- Promote I/O-bound orders, demote CPU-bound
- smoke_break (MLFQ) scores 57.5, but with better tuning could be much higher
- Key question: what quantum lengths and promotion/demotion rules?

B. Better Stride/Proportional Share

- portion_control (stride) has best fairness (0.580) but low completion (81.3%)

- Can stride be combined with SJF for better completion while keeping fairness?
- Need proper implementation tracking per-order virtual time

C. Look-Ahead / Predictive Scheduling

- Track recipe statistics across runs
- Predict which orders will have oven steps and prioritize them
- Use history (obs.history) for better estimation in blind profile

D. Lottery/Proportional Scheduling

- Assign tickets based on priority AND urgency
- Random selection weighted by tickets
- Naturally provides proportional fairness

E. Anti-Starvation with SJF

- Run SJF normally but force-serve the oldest order every N decisions
- Or: maintain a “fairness budget” - when fairness drops too low, switch to FIFO temporarily
- Track Jain’s index in real-time and adapt

F. Work-Until-Wait Optimization

- Orders with short work before an oven step free their cook quickly
- Prioritize these for better I/O overlap
- This helped on bake-off profile but not on default - needs tuning

G. Station-Aware Scheduling

- Identify bottleneck stations (most often full)
- Prefer orders going to underutilized stations
- Avoid orders that will get “bumped” (next station is full)
- Especially important for service-line profile

H. Dynamic Strategy Switching

- Early in the run: aggressive SJF for fast response
- Late in the run: switch to fairness-focused scheduling
- Or: switch based on queue depth / utilization metrics

I. Better Deadline Filtering

- Current: skip if $\text{time_left} < \text{remaining} + \text{switch_cost}$
- Could be: skip if $\text{time_left} < \text{remaining} + \text{switch_cost} + \text{estimated_wait_in_queue}$
- Or: attempt “borderline” orders when no better option exists

10. Available Commands

```
# Run your scheduler
./start.sh play my-scheduler

# Compare with all references
./start.sh compare my-scheduler
```

```

# Evaluate over multiple seeds
./start.sh evaluate my-scheduler --seeds 1000..1020

# Run on a specific profile
./start.sh play my-scheduler --config rush

# Validate submission
./start.sh check my-scheduler

# View replay
./start.sh play my-scheduler --replay

```

11. What I Need From You

Goal: Improve the scheduler score from 68.7 to above 71.0 (beat gordon_ramsay).

Main challenge: Closing the fairness gap (0.435 -> 0.558) without losing too much on response/slowdown.

Constraints: - Single Python file, max 256 KB - 50ms per schedule() call - No external libraries (only kitchen module available) - Must work across 9 different profiles

What would help: 1. A novel scheduling algorithm or combination that balances SJF efficiency with fairness 2. Specific parameter values that work better than what I've tried 3. Profile-specific strategies that significantly boost non-default profiles 4. A way to track and improve Jain's fairness index in real-time 5. Analysis of what gordon_ramsay might be doing differently

12. Project File Structure

```

short-order-student-0.3.0/
|-- my-scheduler/
|   +-- scheduler.py          <-- THE FILE TO EDIT (your submission)
|-- configs/                  <-- 9 marking profiles (.toml files)
|-- student-sdk/kitchen/      <-- API classes (DO NOT EDIT)
|   |-- __init__.py
|   |-- scheduler.py          <-- Base Scheduler class
|   |-- decision.py           <-- Decision class
|   |-- observation.py         <-- Observation, Order, Core, Station classes
|   |-- helpers.py            <-- fill_idle helper
|   +-- ...
|-- engine/linux-x86_64/      <-- Pre-compiled engine (DO NOT EDIT)
|-- docs/student-guide.md     <-- Full documentation
|-- start.sh                  <-- Run command
+-- replays/                  <-- Recorded runs

```

Generated: September 2026 | Engine version: 0.3.0 | Current best score: 68.7 +/- 0.9